Find the Golden Noise

Yuhan Chi

Fudan University

https://yuhanchi.github.io/writing/find-the-golden-noise/

October 18, 2026

Two recent surveys map out reinforcement learning for flow and diffusion models: Nahim, Khan, and Ju's A Survey of Flow Matching in Reinforcement Learning (TMLR, 2026), which organizes the field by *what* the flow models (actions, critics, dynamics) and *where* the RL signal enters (energy-weighted regression, policy gradients through the ODE, GRPO-style objectives), and Ma et al.'s broader Reinforcement Learning-driven Diffusion Models: Language, Vision, and Beyond (2026). Both are useful maps, and the flow-matching survey touches on a few pieces of what follows—such as ReFORM (Zhang et al., 2026) in offline RL, or residual policies that edit an action *after* it is generated. Still, neither survey brings together **steering a frozen generator through its input noise** as a distinct family.

The idea is simple: leave the velocity field untouched, keep the pretrained flow or diffusion policy frozen, and train a small network to decide **which noise to feed it**. Given a state s, a proposer picks a point w in noise space, and the frozen generator turns w into an action. Imagine a robot policy that sometimes reaches around an obstacle on the left and sometimes on the right. Steering the noise changes how often it takes each route while reusing the generator that produces the movements. I like to call this w **the golden noise**. This post follows that thread—along with a few neighbors that also update the decoder or learn from corrections rather than rewards—around two main points:

1. A **VAE-style inference view** (Section 2) connects the methods. The noise proposer plays the role of an encoder and the frozen generator that of a decoder. For a KL-regularized objective, this becomes an exact variational identity, with value taking the place of reconstruction quality. Other methods share the architecture without optimizing that same objective.
2. The most useful question to ask of each method is **where RL takes effect** (Section 3). Action-space data can train a critic directly, but the proposer lives in noise space. Each method is defined largely by how it carries value from one space to the other.

A **two-dimensional experiment** (Section 4) trains the same noise proposer with five different ways of carrying value, so one can watch the noise move and the actions follow. Sections 5–7 then map the broader family, ask what the VAE view explains about its limits, and note a few open questions.

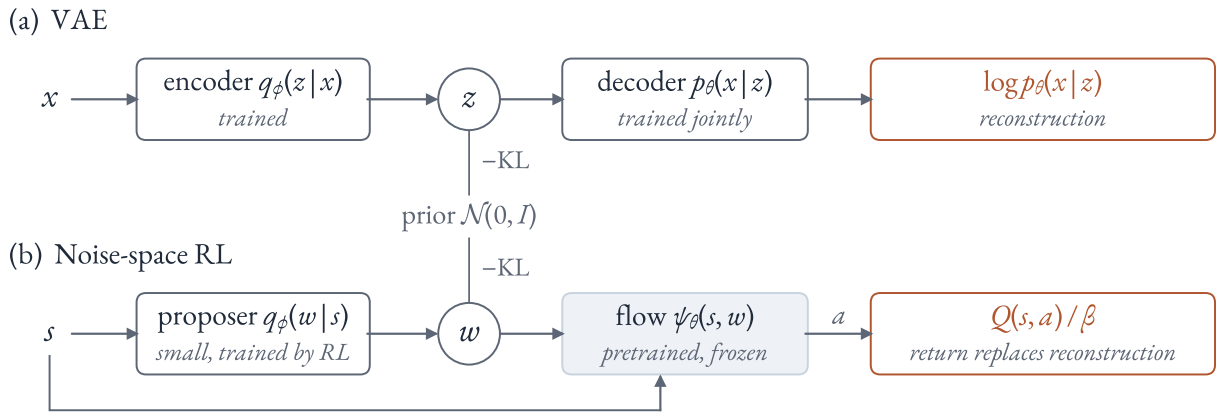


Figure 1: The shared encoder–decoder pattern. (a) A VAE trains an encoder and a decoder jointly to reconstruct its input. (b) In noise-space RL, a proposer q_ϕ chooses the noise w for a state s, and a pretrained, frozen flow ψ_θ decodes it into an action. Both latents are pulled toward the same prior $\mathcal{N}(0, I)$ by a KL term. The decoder in (b) produces an action rather than a reconstruction, and value $Q(s, a)/\beta$ (labeled "return" in the diagram) takes the place of the reconstruction term. The correspondence is between objectives (Section 2), not between generative models.

1 Setup and notation

A flow-matching policy learns a velocity field $v_\theta(x, t, s)$ that transports Gaussian noise at time $t = 0$ to actions at $t = 1$. To act, one draws $w \sim \mathcal{N}(0, I)$ and integrates $\dot{x}_t = v_\theta(x_t, t, s)$ from $x_0 = w$, usually with a few Euler steps,

$$x_{k+1} = x_k + \Delta t \, v_\theta(x_k, t_k, s), \qquad t_k = k\,\Delta t, \quad \Delta t = 1/K, \quad k = 0, \ldots, K-1, \tag{1}$$

and outputs $a = x_K$. Run with any deterministic sampler (this Euler scheme, the probability-flow ODE of a diffusion policy, DDIM with zero variance, or a one-step distilled model), the policy becomes a fixed map

$$a = \psi_\theta(s, w), \qquad w \sim \mathcal{N}(0, I), \tag{2}$$

and all of its randomness enters through w. Table 1 collects the notation used in the rest of the post; the decoder parameters θ are pretrained and stay frozen, and only the proposer parameters ϕ are learned.

A noise-space method keeps θ fixed and replaces only the distribution of the input, from $\mathcal{N}(0, I)$ to $q_\phi(\cdot \mid s)$, turning the base policy $\pi_{\text{base}}(\cdot \mid s) = \psi_\theta(s, \cdot)_\# \mathcal{N}(0, I)$ into the steered policy $\pi_\phi(\cdot \mid s) = \psi_\theta(s, \cdot)_\# q_\phi(\cdot \mid s)$ (where $f_\# \mu$ denotes the distribution of $f(x)$ when $x \sim \mu$). The proposer can be a Gaussian with a state-dependent mean and variance, a deterministic map, a residual added to fresh noise, or even a single constant vector. It is usually smaller than the generator, though not necessarily tiny: Noise PPO uses a roughly 187M-parameter proposer. The decoder can be very large (DSRL steers π_0, a 3.3B-parameter generalist policy) and is never updated.

DSRL states the construction cleanly. Define a new MDP whose actions are noises:

Table 1: Notation.

Symbol	Meaning
$s,\ a$	state and action (for robot policies, often a chunk of future actions)
$w \in \mathbb{R}^d$	noise fed to the generator; for flow policies, d equals the action dimension
$v_\theta,\ K$	velocity field of the pretrained flow, and the number of sampler steps
$\psi_\theta(s, w)$	the **decoder**: the deterministic sampler that maps noise w to action a
$f_\# \mu$	pushforward: the distribution of $f(x)$ when $x \sim \mu$
$\pi_{\text{base}}(\cdot \mid s)$	the pretrained policy, $\psi_\theta(s, \cdot)_\# \mathcal{N}(0, I)$
$q_\phi(w \mid s)$	the **noise proposer** (encoder), usually a state-conditioned Gaussian
$\pi_\phi(\cdot \mid s)$	the steered policy, $\psi_\theta(s, \cdot)_\# q_\phi(\cdot \mid s)$
$Q(s, a)$	an action-value estimate (critic)
$\beta > 0$	temperature: weight of the KL penalty that keeps q_ϕ near $\mathcal{N}(0, I)$
$q^\star,\ \pi^\star$	the optimal tilted noise distribution and policy, defined in Section 2

$$P^W(\cdot \mid s, w) = P(\cdot \mid s, \psi_\theta(s, w)), \qquad r^W(s, w) = r(s, \psi_\theta(s, w)). \tag{3}$$

The diffusion policy becomes part of the environment. In theory, any RL algorithm can now run over w, using only forward passes of the generator, without its weights or its intermediate denoising steps.

Hence, we call w, which the model learns to choose, the golden noise. The idea is older than diffusion policies. PLAS (Zhou et al., 2020) learned a policy in the latent space of a conditional VAE for offline RL. PARROT (Singh et al., 2021) trained a normalizing-flow behavioral prior and ran RL on its noise. It compared directly against a conditional-VAE latent space and argued that invertibility lets the agent keep full control over the final action. DSRL singles out PARROT as the precedent in control and makes the same point from the other side: diffusion models are not invertible by construction, so it is "much less clear" that steering their noise is expressive enough. In practice, it turned out to be.

2 Reading it as a VAE

Recall the evidence lower bound of an ordinary VAE (Kingma & Welling, 2014); a conditional VAE additionally conditions its distributions on context:

$$\log p(x) \ \geq\ \mathbb{E}_{q_\phi(z|x)}\big[\log p_\theta(x \mid z)\big] \ -\ \mathrm{KL}\big(q_\phi(z \mid x) \,\|\, p(z)\big). \tag{4}$$

To make the connection precise, consider the following KL-regularized policy-improvement objective:

$$J(\phi) = \mathbb{E}_s\Big[\,\mathbb{E}_{w \sim q_\phi(\cdot|s)}\, Q\big(s, \psi_\theta(s, w)\big) \ -\ \beta\, \mathrm{KL}\big(q_\phi(\cdot \mid s) \,\|\, \mathcal{N}(0, I)\big)\Big]. \tag{5}$$

Here Q is held fixed during the proposer update, and states are sampled from a fixed training distribution. This is a local improvement objective, not by itself a complete solution to sequential RL. Table 2 lines up the two expressions term by term.

Table 2: VAE terms and their noise-space counterparts.

VAE	Noise-space RL
observed input x	desired optimality, with state s as context
encoder $q_\phi(z \mid x)$	noise proposer $q_\phi(w \mid s)$
prior $p(z) = \mathcal{N}(0, I)$	the Gaussian the flow was trained from
decoder $p_\theta(x \mid z)$	frozen flow $\psi_\theta(s, w)$
reconstruction $\log p_\theta(x \mid z)$	value $Q(s, \psi_\theta(s, w))/\beta$
KL to the prior	KL to $\mathcal{N}(0, I)$: penalize departure from the training prior
decoder trained jointly	decoder pretrained by imitation, usually frozen

The exact connection is a variational identity. Define $Z(s) = \mathbb{E}_{w\sim\mathcal{N}(0,I)}[\exp(Q(s, \psi_\theta(s, w))/\beta)]$ and assume it is finite. Then, for each state,

$$\log Z(s) = \underbrace{\mathbb{E}_{q_\phi}[Q(s, \psi_\theta(s, w))/\beta] - \mathrm{KL}(q_\phi \| \mathcal{N}(0, I))}_{J(\phi; s)/\beta} + \mathrm{KL}(q_\phi \| q^\star). \tag{6}$$

The last term is nonnegative, so the first two terms form a lower bound. In the control-as-inference view (Levine, 2018), this can be read as an ELBO for a desired "optimality" observation. When Q is bounded above, a state-dependent scaling makes the exponential a valid binary likelihood; the variational identity itself needs only a finite $Z(s)$. The unrestricted best proposer is

$$q^\star(w \mid s) = \frac{\mathcal{N}(w; 0, I)\, \exp\big(Q(s, \psi_\theta(s, w))/\beta\big)}{Z(s)}. \tag{7}$$

Derivation of the variational identity. Write $p_0(w) = \mathcal{N}(w; 0, I)$ and suppress the fixed state s. Substituting the definition of $q^\star$ into $\mathrm{KL}(q_\phi \| q^\star)$ and splitting the logarithm gives

$$\begin{aligned}
\mathrm{KL}(q_\phi \| q^\star) &= \mathbb{E}_{w\sim q_\phi}\left[\log \frac{q_\phi(w)}{p_0(w)\, \exp(Q(\psi_\theta(w))/\beta)/Z}\right] \\
&= \mathbb{E}_{w\sim q_\phi}\left[\log \frac{q_\phi(w)}{p_0(w)}\right] - \mathbb{E}_{w\sim q_\phi}\big[Q(\psi_\theta(w))/\beta\big] + \log Z \\
&= \mathrm{KL}\big(q_\phi \| \mathcal{N}(0, I)\big) - \mathbb{E}_{q_\phi}\big[Q(\psi_\theta(w))/\beta\big] + \log Z.
\end{aligned}$$

Rearranging terms yields $\log Z = \mathbb{E}_{q_\phi}[Q(\psi_\theta(w))/\beta] - \mathrm{KL}(q_\phi \| \mathcal{N}(0, I)) + \mathrm{KL}(q_\phi \| q^\star)$. □

Writing $J(\phi; s)$ for the term inside the outer expectation of J, the identity also gives a ruler: for any proposer, $\log Z(s) - J(\phi; s)/\beta = \mathrm{KL}(q_\phi \| q^\star)$ measures how far it is from the best one. Section 4 uses it.

The difference from an ordinary VAE is what counts as the observation. The encoder is not asked to explain an action that was seen. It is asked to infer which noise *would have produced a good action*. The evidence is desired success, represented here by the value estimate. This is why "VAE-style" is more accurate than calling every method a conditional VAE. A deterministic

proposer, for example, has infinite KL to a continuous Gaussian; box constraints, spherical latents, and supervised inversion require their own objectives.

Two facts make this reading useful rather than decorative. Eyring et al. (2025), in Noise Hypernetworks, state both for distilled text-to-image generators, and both carry over to policies without change.

Proposition 1 (Tilting the noise tilts the policy)**.** *Pushing the noise posterior through the frozen decoder gives exactly the reward-tilted base policy:*

$$\psi_\theta(s,\cdot)_\# q^\star(\cdot \mid s) \;=\; \pi^\star(\cdot \mid s) \;\propto\; \pi_{base}(\cdot \mid s)\, \exp\big(Q(s,\cdot)/\beta\big). \tag{8}$$

The reason is that the tilt depends on w only through $a = \psi_\theta(s,w)$.

Proof of Proposition 1. For a bounded test function f, suppressing the fixed state s,

$$\begin{aligned}\mathbb{E}_{q^\star}[f(\psi_\theta(w))] &= Z^{-1}\mathbb{E}_{w\sim\mathcal{N}(0,I)}[f(\psi_\theta(w))e^{Q(\psi_\theta(w))/\beta}] \\ &= Z^{-1}\mathbb{E}_{a\sim\pi_{\text{base}}}[f(a)e^{Q(a)/\beta}].\end{aligned}$$

□

No invertibility is needed. This is the same target that KL-regularized fine-tuning of the whole flow aims for, but a small proposer can only approximate it as well as its distribution family and optimization allow. A single Gaussian, for example, need not represent a multimodal noise posterior. Figure 2 shows the target on a 2D example.

Proposition 2 (The noise-space KL is a conservative proxy)**.** *By the data processing inequality,*

$$\mathrm{KL}\big(\pi_\phi(\cdot \mid s) \,\|\, \pi_{base}(\cdot \mid s)\big) \;\le\; \mathrm{KL}\big(q_\phi(\cdot \mid s) \,\|\, \mathcal{N}(0,I)\big), \tag{9}$$

with equality when $\psi_\theta(s,\cdot)$ is invertible. The left side can be expensive to evaluate: a continuous normalizing flow can compute densities through divergence integration, while a generic distilled decoder need not offer a tractable density. For a Gaussian proposer $\mathcal{N}(\mu, \mathrm{diag}\,\sigma^2)$, the right side is the familiar VAE term $\frac{1}{2}\sum_j(\mu_j^2 + \sigma_j^2 - 1 - \log\sigma_j^2)$. Noise PPO (Miao et al., 2025) uses exactly this KL penalty, together with an entropy bonus. Noise Hypernetworks (Eyring et al., 2025) and LP-DS (Simsir & Oguz, 2026) write the steered noise as a residual $w = \epsilon + \Delta$ and approximate the KL by $\frac{1}{2}\,\mathbb{E}\|\Delta\|^2$, a surrogate justified under regularity and small-Jacobian assumptions. Bounding the residual's magnitude alone does not establish those assumptions.

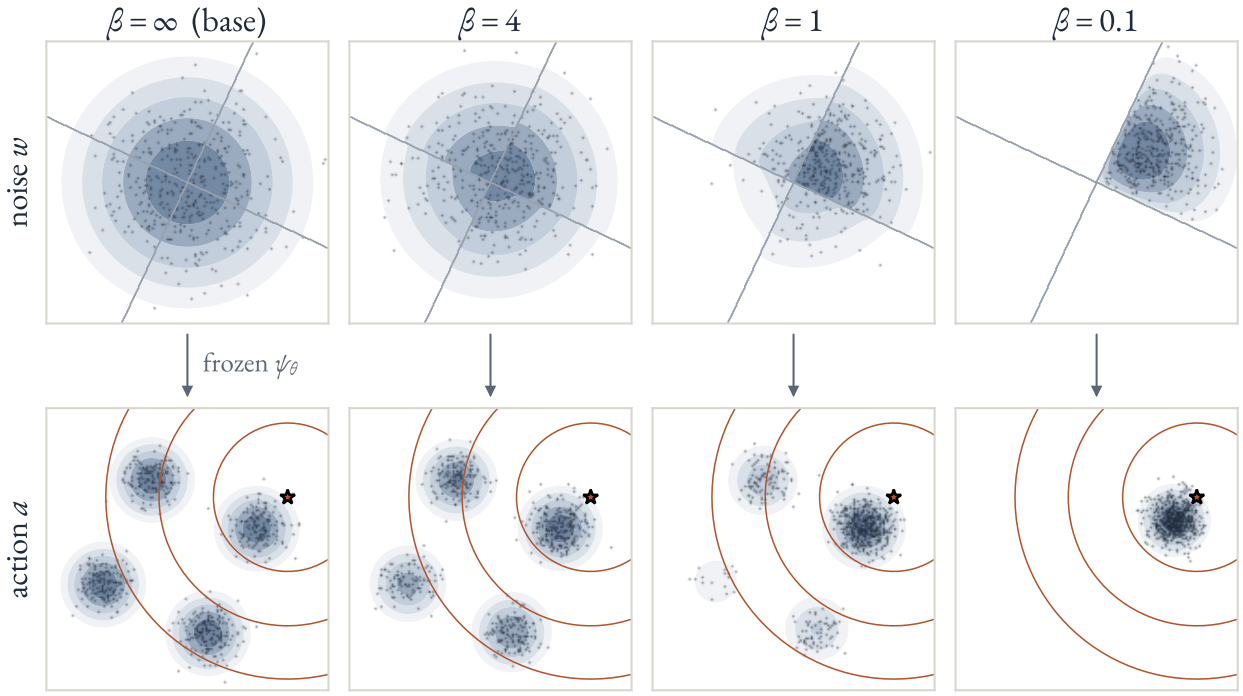


Figure 2: Tilting the noise tilts the policy (exact, no learning). The base policy is a four-cluster mixture in a 2D action space, and ψ_θ is the exact flow-matching ODE map onto it. Each column is one value of β. Top: the tilted noise distribution $q^\star(w) \propto \mathcal{N}(w; 0, I)\, e^{Q(\psi_\theta(w))/\beta}$ (shading) and samples from it (dots); grey lines split the noise plane into the four regions that decode to the four clusters. Bottom: the same samples after the frozen flow, over the tilted policy $\pi_{\text{base}}\, e^{Q/\beta}$ (shading); dashed circles are contours of Q, and the star marks its peak. At $\beta = \infty$ we recover the base policy. As β shrinks, probability moves to the cluster nearest the peak, but no new cluster appears.

3 Where RL takes effect: carrying value into noise space

The decision variable for RL is the noise w, whereas rewards, demonstrations, and human corrections all live in action space. Training the proposer therefore comes down to carrying signal across the composition $w \mapsto Q\big(s, \psi_\theta(s, w)\big)$. Depending on whether we use forward calls, an inverse, or derivatives of ψ_θ, the literature takes five routes (Figure 3).

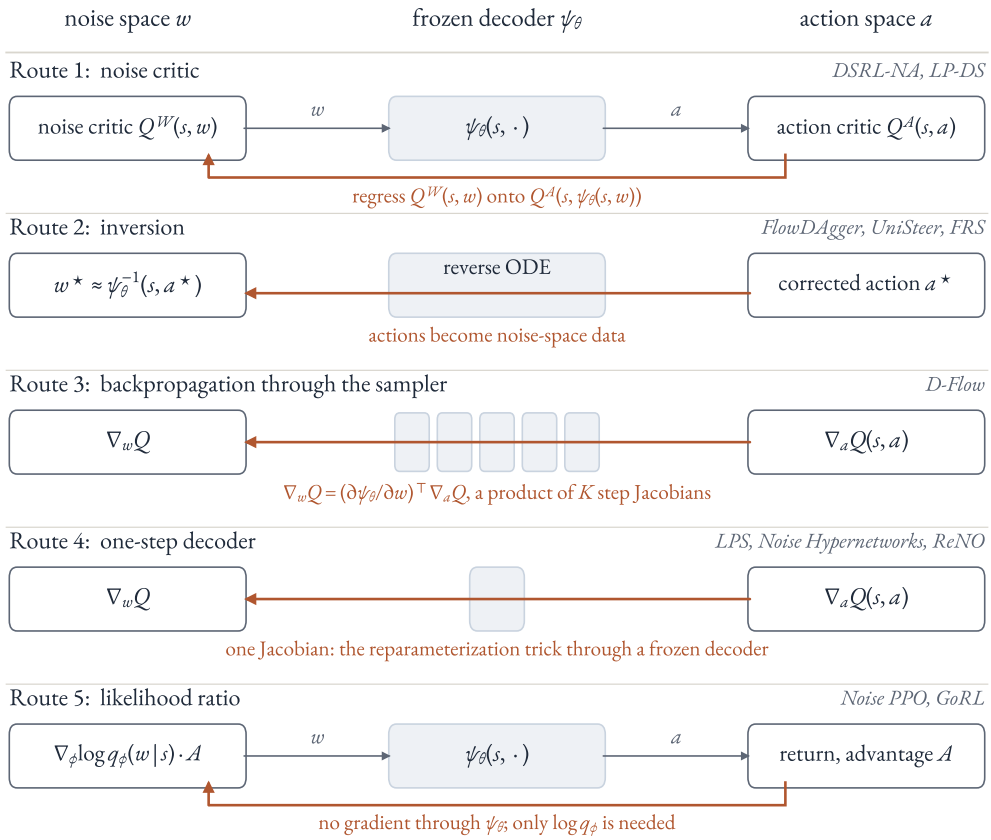


Figure 3: Five routes for training the noise proposer. In each row, noise space is on the left, the frozen decoder in the middle, and action space on the right. Grey dashed arrows are forward decoding; orange arrows show what crosses back into noise space. Route 1 is drawn in its two-critic form; Route 2 uses action supervision rather than a critic; Route 5 uses returns directly. Representative methods are listed at the right of each row.

Route 1: learn a critic in noise space. Train an action critic $Q^A(s,a)$ by standard TD learning, distill it into a noise critic $Q^W(s,w)$ using forward decoder calls only (DSRL-NA, Wagenmaker et al., 2025; LP-DS), and update the proposer along $\nabla_w Q^W$:

$$\min_{Q^W} \mathbb{E}_{s,\,w\sim\mathcal{N}(0,I)}\Big[\big(Q^W(s,w) - Q^A(s,\psi_\theta(s,w))\big)^2\Big], \qquad \max_{\phi} \mathbb{E}_{s,\,w\sim q_\phi(\cdot|s)}\big[Q^W(s,w)\big]. \tag{10}$$

DSRL-SAC instead learns Q^W directly from rollouts, and GoldenStart (Zhang et al., 2026) scores candidate noises with Q^A to fit a conditional VAE prior. *Cost:* critic approximation error—fitting values well does not guarantee an accurate gradient $\nabla_w Q^W$ (Im et al., 2026).

Route 2: invert the flow and move the data. When supervision is a target action $a^\star$ (human corrections in FlowDAgger, Murray et al., 2026, and UniSteer, Lu et al., 2026; or coarse references in Flow Reversal Steering, Tang et al., 2026), backward ODE integration recovers a target noise $w^\star$ for supervised learning:

$$w^\star = \psi_\theta^{-1}(s, a^\star) \quad \big(x_k = x_{k+1} - \Delta t\, v_\theta(x_k, t_k, s)\big), \qquad \max_\phi \; \mathbb{E}_{(s,a^\star)}\big[\log q_\phi(w^\star \mid s)\big]. \tag{11}$$

Cost: inversion compute and error—short ODE schedules make naive reverse Euler inaccurate, requiring per-step fixed-point iteration.

Route 3: backpropagate through the sampling iterations. With reparameterization $w = \mu_\phi(s) + \sigma_\phi(s) \odot \epsilon$, the chain rule differentiates straight through the K-step Euler solver (D-Flow, Ben-Hamu et al., 2024):

$$\nabla_w Q\big(s, \psi_\theta(s, w)\big) = \Big(\frac{\partial \psi_\theta}{\partial w}\Big)^\top \nabla_a Q(s, a), \qquad \frac{\partial \psi_\theta}{\partial w} = \prod_{k=K-1}^{0} \Big(I + \Delta t\, \frac{\partial v_\theta}{\partial x}(x_k, t_k, s)\Big). \tag{12}$$

Cost: backpropagating through all K solver steps is memory-intensive and numerically unstable for multi-step policies.

Route 4: make the decoder one step. For a one-step generator ($K = 1$, $a = w + v_\theta(w, 0, s)$), the Jacobian product in Route 3 collapses to a single factor (LPS, Im et al., 2026; ReNO, Eyring et al., 2024; Noise Hypernetworks, Eyring et al., 2025):

$$\nabla_w Q\big(s, \psi_\theta(s, w)\big) = \Big(I + \frac{\partial v_\theta}{\partial w}(w, 0, s)\Big)^\top \nabla_a Q(s, a). \tag{13}$$

Cost: requires training or distilling a one-step generator first.

Route 5: skip the transfer. Treating the decoder and critic as a black box, the score-function (REINFORCE / PPO) gradient updates q_ϕ from scalar advantages $A(s, w)$ alone (Noise PPO, Miao et al., 2025; GoRL, Zhang et al., 2025):

$$\nabla_\phi \mathbb{E}_{w \sim q_\phi(\cdot|s)}\big[Q(s, \psi_\theta(s, w))\big] = \mathbb{E}_{w \sim q_\phi(\cdot|s)}\Big[\nabla_\phi \log q_\phi(w \mid s)\, A(s, w)\Big]. \tag{14}$$

For a Gaussian proposer, $\log q_\phi(w \mid s)$ is closed-form, bypassing ODE divergence integrals entirely. *Cost:* high score-function gradient variance.

4 A two-dimensional experiment

To see the five routes at work, I trained the same Gaussian proposer $q_\phi(w \mid s) = \mathcal{N}(\mu_\phi(s), \operatorname{diag} \sigma_\phi(s)^2)$ on a 2D toy where $q^\star$ is known (code on GitHub). A frozen $K = 10$-step Euler flow ψ_θ maps $\mathcal{N}(0, I)$ to the four clusters of Figure 2, and each state $s \in [0, 2\pi)$ sets a goal $g(s)$ with exact critic $Q(s, a) = -\frac{1}{2}\|a - g(s)\|^2$ ($\beta = 1$). In Figure 4, the goal sits between

two clusters so the target policy is bimodal in action space, yet a single Gaussian proposer can straddle both because they decode from adjacent wedges of noise space (**a bimodal action target can be unimodal in noise space**).

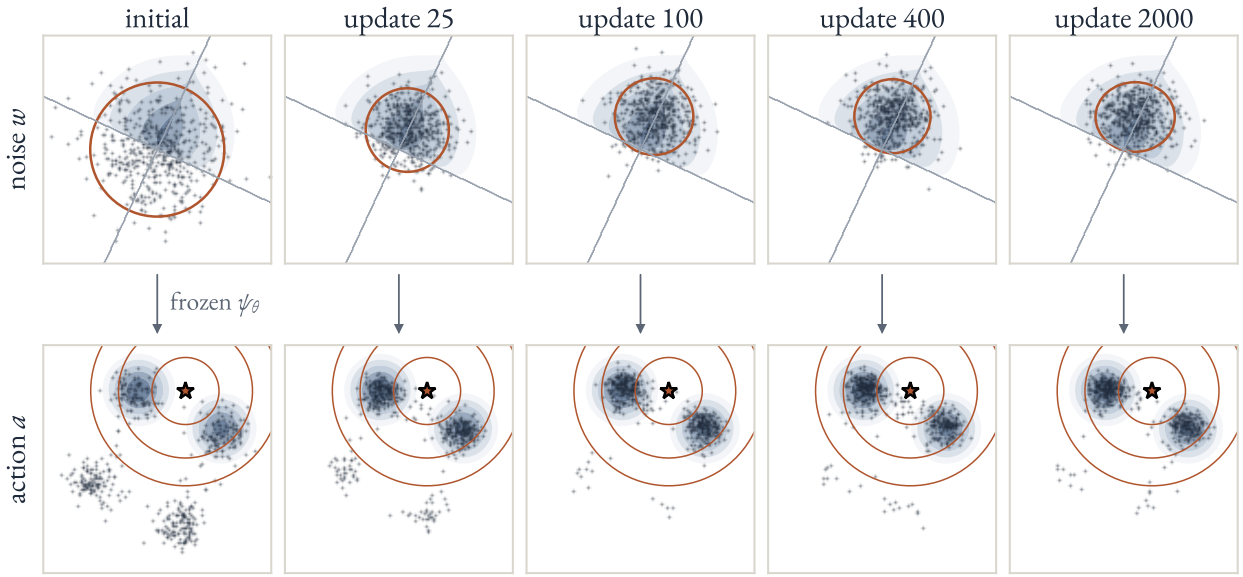


Figure 4: Learning which noise to choose, for one state (goal angle 67.5°) with Route 3. Columns are snapshots during training. Top: samples from the proposer $q_\phi(w \mid s)$ (dots) and its two-standard-deviation ellipse (orange), over the exact tilted posterior $q^\star$ for the learned decoder (shading); grey lines separate the noise regions that decode to different clusters. Bottom: the same samples after the frozen 10-step decoder, over the tilted target policy $\pi_{\text{base}}\, e^{Q/\beta}$ (shading, computed from the mixture the flow imitates); the star is the goal and dashed circles are contours of Q. The decoder never changes; only the noise does.

Routes 1, 3, 4, and 5 all converge to roughly the same point (0.206–0.211 nats, close to the single-Gaussian ceiling of 0.198), differing mainly in their trajectories and speed (Route 4 takes 2.9 ms per update; Route 1 lags early on while its noise critic warms up).

Route 2 settles higher (0.312), not because fixed-point inversion fails (its reconstruction error is $\sim 10^{-7}$), but because it optimizes the forward KL rather than the reverse KL:

$$\text{Routes 1, 3, 4, 5 (RL):} \quad \min_\phi \mathrm{KL}(q_\phi \| q^\star) = \mathbb{E}_{w\sim q_\phi}\left[\log \frac{q_\phi(w \mid s)}{q^\star(w \mid s)}\right], \tag{15}$$

$$\text{Route 2 (MLE on } w^\star \sim q^\star\text{):} \quad \min_\phi \mathrm{KL}(q^\star \| q_\phi) = \mathbb{E}_{w^\star\sim q^\star}\left[\log \frac{q^\star(w^\star \mid s)}{q_\phi(w^\star \mid s)}\right]. \tag{16}$$

Reverse KL is *mode-seeking*: it penalizes spilling across the grey boundaries into low-value wedges and pulls the Gaussian tight. Forward KL is *mode-covering*: maximum likelihood stretches the Gaussian to cover all inverted samples (Figure 5a), spilling into adjacent low-value regions.

Table 3: Five routes on the 2D toy (2,000 updates, 256 states $\times$ 4 noises per update). $\mathrm{KL}(q_\phi \| q^\star) = \log Z(s) - J(\phi; s)/\beta$ is averaged over 64 states (mean $\pm$ standard deviation over three seeds; lower is better). Time is the median wall-clock time per update on an RTX 5060 GPU. Route 4 is measured against the posterior of its own one-step decoder.

Route	Needs from the decoder	$\mathrm{KL}(q_\phi \| q^\star)$	ms / update
1. noise critic	forward calls	0.207 ± 0.004	8.6
2. inversion	an inverse (20 fixed-point iterations per step)	0.312 ± 0.007	60.2
3. backprop through 10 steps	derivatives	0.211 ± 0.002	7.2
4. one-step decoder	derivatives of a distilled one-step student	0.206 ± 0.002	2.9
5. likelihood ratio	forward calls	0.211 ± 0.003	5.6
reference: best Gaussian for each state separately	derivatives	0.198	—

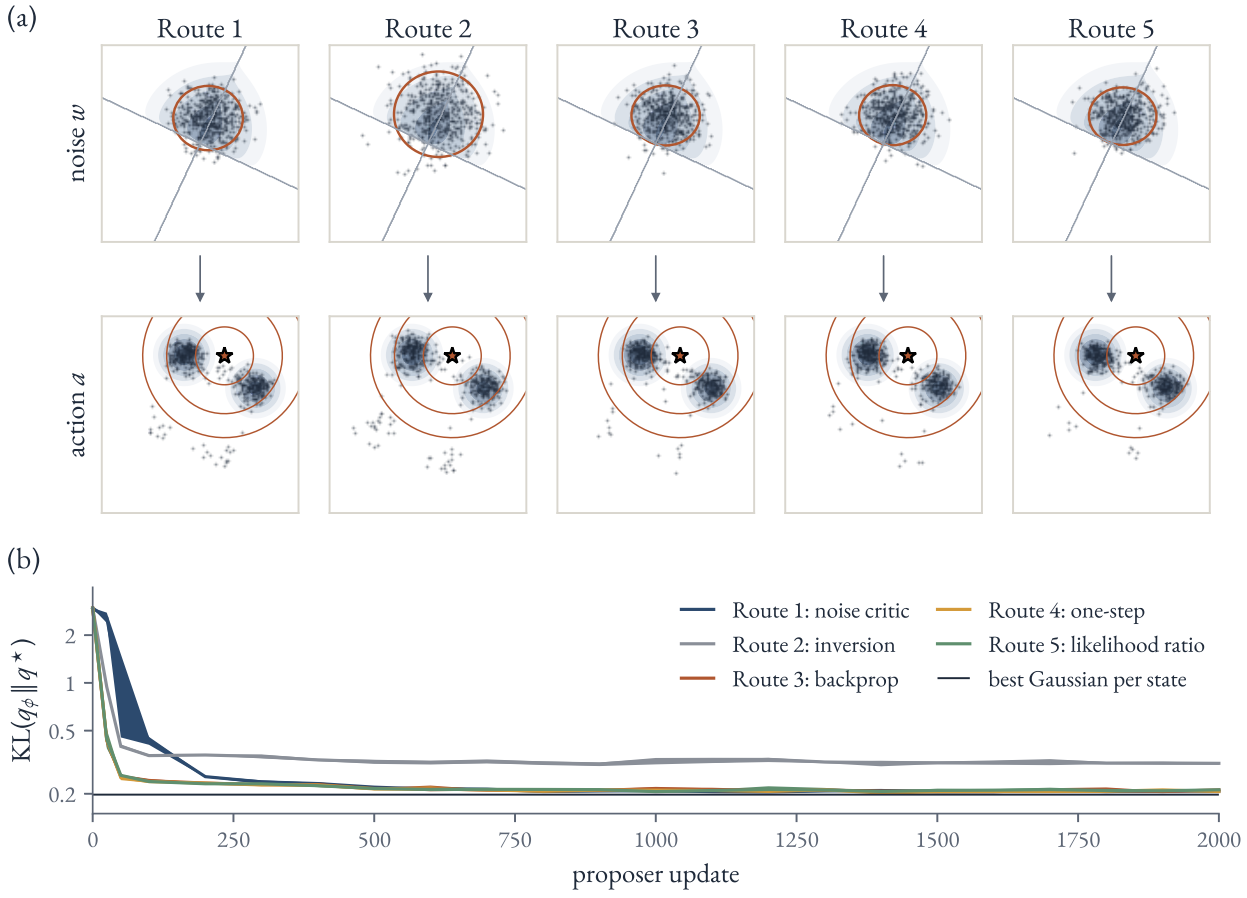


Figure 5: Five routes, one proposer. (a) The final proposer of each route for the same state as Figure 4, drawn as in Figure 4 (seed 0). (b) $\mathrm{KL}(q_\phi \| q^\star)$ averaged over 64 states during training; lines are means and bands span three seeds. The dashed line is the best single Gaussian fitted to each state separately.

5 A map of the family

Table 4 groups the methods by their main learning route. It includes related designs with changing decoders, labeled explicitly. "Prior control" distinguishes actual constraints from initialization or supervision alone.

Table 4: The family at a glance, grouped by route.

Route	Method	Setting	Proposer $q_\phi(w \mid s)$	Prior control
1. noise critic	PARROT (2021; preprint 2020)	robot manipulation	RL policy over the noise of an invertible normalizing flow	prior initialization; no lasting proximity guarantee
1. noise critic	DSRL (2025)	online and offline control; π_0	state-conditioned noise actor; SAC, or an action critic distilled into a noise critic	noise box $[-b, b]^d$
1. noise critic	LP-DS (2026)	online control; π_0	residual $\epsilon + \Delta_\phi(s, \epsilon)$ with noise-aliased critics	Lagrangian trust region on $\mathbb{E}\|\Delta\|^2$
1. forward scoring	GoldenStart (2026)	distilled one-step flow policies; teacher not frozen	conditional VAE fit to the highest-value noise per state	CVAE fitting; entropy control is on the action student
2. inversion	FlowDAgger, UniSteer (2026)	human-in-the-loop VLA adaptation	noise policy regressed onto inverted corrections; UniSteer adds a noise critic	inverted targets; inversion alone does not ensure typical noise
2. inversion	Flow Reversal Steering (2026)	generalist policies	auxiliary noise policy trained on inverted coarse actions	coarse references guide reversal; no generic KL bound
3 / 4. back-propagation	ReFORM (2026)	offline RL; BC model and distilled decoder also trained	reflected flow generating bounded input noise	projected steps keep noise inside the source ball
4. one-step	Noise Hypernetworks (2025)	distilled text-to-image	residual on fresh noise (LoRA), reward gradient through the generator	$\frac{1}{2}\|\Delta\|^2$ as a KL surrogate
4. one-step	LPS (2026)	offline RL; real robots; base also trained by imitation	latent actor on a sphere, $\nabla_a Q$ through a one-step MeanFlow	sphere of radius $\sqrt{d}$
5. likelihood ratio	Noise PPO (2025)	text-to-image	prompt-conditioned Gaussian trained by PPO	KL to $\mathcal{N}(0, I)$, entropy bonus
5. likelihood ratio	GoRL (2025)	online control	Gaussian encoder; decoder refit in alternating phases	KL to $\mathcal{N}(0, I)$
search	Golden Ticket (2026)	generative robot policies	one constant noise vector found by search	—

ReFORM spans Routes 3 and 4: its original objective differentiates through the BC flow, and its practical design introduces one-step BC distillation to shorten this path. Its Algorithm 1 also continues fitting the BC model.

Three neighbors are worth knowing. PDP (Zhang et al., 2026) argues that Gaussian noise is a poorly conditioned control interface. Instead of steering it, PDP trains a literal VAE over whole demonstrations, with a geometry-alignment term, and modulates the denoiser with the resulting low-dimensional code. Condition-dependent source flow matching (Kim et al., 2026) and LAFM (Machado et al., 2026) learn the prior itself together with the flow at training time, which in VAE

terms is a learned prior. D-Flow (Ben-Hamu et al., 2024) and ReNO (Eyring et al., 2024) optimize the noise separately for each sample instead of training a proposer.

6 What the VAE view explains

Support and latent holes. A frozen decoder restricts actions to the image of $\psi_\theta(s, \cdot)$, and for finite Q and $\beta > 0$ the tilted policy shares the base policy's support. Mathematical support, however, is not a safety guarantee: just as a VAE decodes poorly in low-density holes of its prior, a frozen flow breaks when queried outside typical Gaussian regions. Different methods keep w out of those holes in different ways—clipping to $[-b, b]^d$ (DSRL), restricting w to the $\sqrt{d}$ sphere (LPS), bounding $\mathbb{E}\|\Delta\|^2$ (LP-DS), or bounding the source ball (ReFORM)—while Golden Ticket (Patil et al., 2026) uses a single state-independent vector and GoRL (Zhang et al., 2025) updates the decoder itself.

Aliasing. Distinct noises often decode to similar actions. DSRL-NA turns this redundancy into sample efficiency by sharing Q^A values across unvisited noises, while in the data processing inequality $\mathrm{KL}(\pi_\phi \| \pi_{\text{base}}) \leq \mathrm{KL}(q_\phi \| \mathcal{N}(0, I))$, aliasing explains the gap: KL budget spent redistributing probability among noises that produce identical actions does not change the action distribution.

Amortization cost. Optimizing w per sample at test time (D-Flow, ReNO) versus training a proposer $q_\phi(w \mid s)$ (Noise Hypernetworks) mirrors per-datapoint variational inference versus an amortized encoder: amortization trades an optimality gap (Cremer et al., 2018) for a single forward pass at deployment.

7 Open questions and takeaway

- **How should the encoder–decoder pair be trained: decoupled or jointly?** Almost every method here freezes the pretrained decoder and only updates the noise encoder; GoRL is a rare exception that alternates between the two. Whether decoupled or joint training works better—and what update ratio between encoder and decoder gives the best performance—has barely been explored.
- **Where is the Pareto frontier among the five routes?** Each route has clear trade-offs: critic distillation (Route 1) risks gradient mismatch, inversion (Route 2) minimizes the forward KL at high per-step cost, multi-step backpropagation (Route 3) is heavy and unstable, one-step distillation (Route 4) alters the base model, and likelihood-ratio updates (Route 5) suffer from high variance. Finding a Pareto-optimal recipe for RL fine-tuning remains hard.
- **How much steerability can initial noise alone provide?** Diffusion and flow policies owe much of their multimodal expressiveness to dense, multi-step denoising. If adaptation is restricted to choosing the initial noise $x_0 = w$—and w cannot stray far from the prior without hitting latent holes—how much steerability does that actually leave, and when is steering x_0 simply too weak?

The flow-matching survey's second axis asks where the RL signal enters a flow-matching method. This family adds one more answer to that question: **the RL signal can enter through the input-noise distribution.**

References

Ben-Hamu, H., Puny, O., et al. (2024). D-Flow: Differentiating through Flows for Controlled Generation. *ICML*.

Cremer, C., Li, X., & Duvenaud, D. (2018). Inference Suboptimality in Variational Autoencoders. *ICML*.

Eyring, L., Karthik, S., Roth, K., Dosovitskiy, A., & Akata, Z. (2024). ReNO: Enhancing One-step Text-to-Image Models through Reward-based Noise Optimization. *NeurIPS*.

Eyring, L., Karthik, S., Dosovitskiy, A., Ruiz, N., & Akata, Z. (2025). Noise Hypernetworks: Amortizing Test-Time Compute in Diffusion Models.

Im, H., Kolobov, A., Fu, J., & Lee, Y. (2026). Latent Policy Steering through One-Step Flow Policies. *RSS*. (LPS)

Kim, J., Park, J., Jeon, S., & Kim, S. (2026). Better Source, Better Flow: Learning Condition-Dependent Source Distribution for Flow Matching.

Kingma, D. P., & Welling, M. (2014). Auto-Encoding Variational Bayes. *ICLR*.

Levine, S. (2018). Reinforcement Learning and Control as Probabilistic Inference: Tutorial and Review.

Lu, J., Qin, X., et al. (2026). UniSteer: Unified Noise Steering for Efficient Human-Guided VLA Adaptation.

Ma, Z., Wei, J., Wang, T., et al. (2026). Reinforcement Learning-driven Diffusion Models: Language, Vision, and Beyond.

Machado, B., Chapin, A., Dellandréa, E., & Chen, L. (2026). Flowing With Purpose: Latent Action Guided Flow Matching Policies for Robotic Manipulation. (LAFM)

Miao, Y., Loh, W., Poupart, P., & Kothawade, S. (2025). A Minimalist Method for Fine-tuning Text-to-Image Diffusion Models. (Noise PPO)

Murray, M., Chen, D., Bagaria, S., et al. (2026). FlowDAgger: Human-in-the-Loop Adaptation of Generative Robot Policies in Latent Space.

Nahim, N. Z., Khan, F. N., & Ju, P. (2026). A Survey of Flow Matching in Reinforcement Learning. *TMLR*.

Patil, O., Biza, O., Weng, T., et al. (2026). You've Got a Golden Ticket: Improving Generative Robot Policies With A Single Noise Vector.

Simsir, H., & Oguz, O. S. (2026). Lagrangian Perturbation Diffusion Steering: Latent Reinforcement Learning for Generative Policies. *ICML*. (LP-DS)

Singh, A., Liu, H., Zhou, G., Yu, A., Rhinehart, N., & Levine, S. (2021). PARROT: Data-Driven Behavioral Priors for Reinforcement Learning. *ICLR*.

Tang, A., Chen, W., Wagenmaker, A., Finn, C., & Levine, S. (2026). Improving Robotic Generalist Policies via Flow Reversal Steering.

Wagenmaker, A., Nakamoto, M., Zhang, Y., et al. (2025). Steering Your Diffusion Policy with Latent Space Reinforcement Learning. (DSRL)

Zhang, C., Wan, Z., Chen, F., Yu, X., Tsang, I., & An, B. (2025). GoRL: An Algorithm-Agnostic Framework for Online Reinforcement Learning with Generative Policies. The current arXiv version is titled *Training Diffusion Policies via Prior-Mapping Co-Evolution*; this post follows v1.

Zhang, H., Sun, Y., & Xiong, H. (2026). GoldenStart: Q-Guided Priors and Entropy Control for Distilling Flow Policies. *ICLR*.

Zhang, R., Fu, H., Jia, M., Konidaris, G., Du, Y., & da Silva, B. C. (2026). From Noise to Control: Parameterized Diffusion Policies. *ICML*. (PDP)

Zhang, S., So, O., Ahmad, H. M. S., Yu, E. Y., Cleaveland, M., Black, M., & Fan, C. (2026). ReFORM: Reflected Flows for On-Support Offline RL via Noise Manipulation. *ICLR*.

Zhou, W., Bajracharya, S., & Held, D. (2020). PLAS: Latent Action Space for Offline Reinforcement Learning. *CoRL*.

Cite this post

If you find this post useful in your work, you can cite it as:

> Chi, Yuhan. "Find the Golden Noise." October 18, 2026. https://yuhanchi.github.io/writing/find-the-golden-noise/

Or use the following BibTeX entry:

```
@misc{chi2026findthegoldennoise,
  author = {Chi, Yuhan},
  title = {{Find the Golden Noise}},
  year = {2026},
  month = oct,
  day = {18},
  howpublished = {Blog post},
  url = {https://yuhanchi.github.io/writing/find-the-golden-noise/}
}
```

This citation refers to the blog's explanation and synthesis. Please cite the original papers above for their methods and results.